PROJECT BLUEPRINT: PHYSICAL AI & SMART BUILDING GATEWAY

Intern Proof-of-Concept (PoC) Implementation Guide

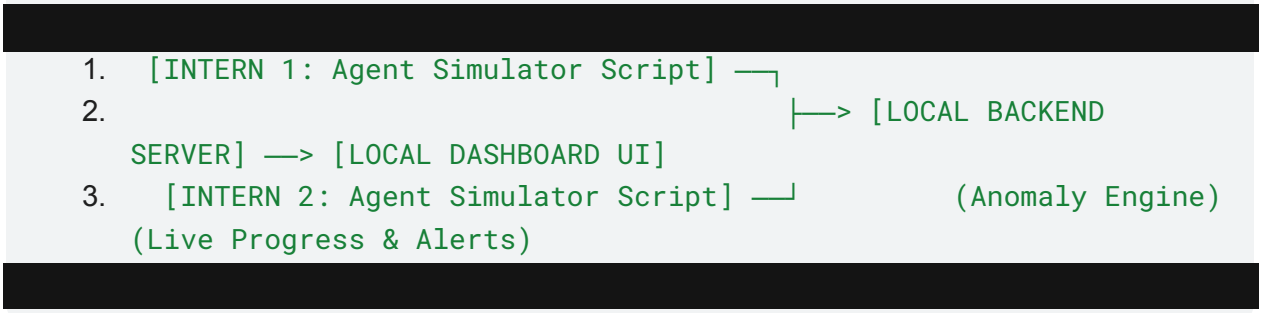
Welcome to your technical challenge! This document outlines your project scope, technical expectations, and clear milestones. You will be collaborating to build a localized **Physical AI & Intelligent Building Management System (BMS)** dashboard from scratch using open-source, zero-cost utilities.

1. THE VISION: WHAT WE ARE BUILDING

Modern building management integrates autonomous physical agents with legacy backend systems. Drones handle high-value exterior thermal/visual telemetry, while ground robots perform interior patrols, automated cleaning, and path monitoring.

Your goal is to build an interactive **Command Center Dashboard** that ingests mock data streams from these agents, identifies anomalies in real-time, and automatically generates maintenance work orders.

```
None
```



2. SETTING UP YOUR ENVIRONMENT (100% FREE & LOCAL)

Because you are working outside the corporate network with no access to company resources, your entire stack must be hosted locally on your personal computers. **Choose one of the following stack options to build upon:**

Option A: The Full-Stack JavaScript Route

- **Code Editor:** Visual Studio Code (Free).
- **Runtime & Server: Node.js (LTS)** using the Express framework to create a REST API backend.
- **Database: MongoDB Community Edition** to log incoming telemetry data and work orders.

Option B: The Classic Web Architecture Route

- **All-in-One Local Suite: XAMPP** or **WampServer** (Free, one-click installer). This instantly gives you a local Apache web server, local PHP environment, and a local **MySQL** database.

Team Collaboration Tool

- **Git & GitHub:** Create a free private personal GitHub repository. Work out of separate feature branches (`feature/agent-stream-1` and `feature/agent-stream-2`) and merge your code using Pull Requests.

3. BRAINSTORMING SCRIPTS: DAILY FACILITY SCENARIOS

Do not limit your thinking to high-end, out-of-reach hardware like military drones or science-fiction humanoids. Physical AI is all around us in the everyday infrastructure of a commercial office building.

To achieve this PoC, **one intern will own Scenario A, and the second intern will own Scenario B.** Choose from the everyday building automation pairs below, or pitch your own daily facility variation:

Track 1: Automated Janitorial & Mopping Robots

- **The Real-World Concept:** Autonomous floor scrubbers and mopping robots operating on night/weekend cleaning schedules.
- **The Telemetry Payload:** Write a background script that uses a timer (`setInterval`) to broadcast an HTTP POST request to your local server every 5 seconds, tracking fields like `water_tank_level_pct`, `battery_life`, and `area_cleaned_sqft`.
- **The Anomaly Trigger:** Program a toggle switch in your script to simulate a failure event—such as running completely out of water mid-floor, a critical low battery, or detecting a persistent liquid spill it cannot clear.
- **The Expected BMS Action:** The central system flags the anomaly, updates the dashboard status, and generates a maintenance work order.

Track 2: Smart HVAC & Ambient Lighting Optimization

- **The Real-World Concept:** Environmental micro-sensing loops that automatically dim building zones or shift HVAC thresholds based on live human occupancy.
- **The Telemetry Payload:** Write your script to stream continuous ceiling-mounted motion and temperature sensor data, monitoring `occupancy_detected (True/False)`, `ambient_temp_celsius`, and `power_consumption_watts`.
- **The Anomaly Trigger:** Program an event trigger representing a critical threshold breach (e.g., ambient room temperature spiking to 29°C due to an AC vent failure while a zone is marked as fully occupied).
- **The Expected BMS Action:** The system immediately outputs an automated cooling correction work order.

Track 3: Facility Logistics & Washroom Asset Monitoring

- **The Real-World Concept:** Smart building facilities tracking supply levels (soap, paper towels) and automated plumbing infrastructure to optimize janitorial routes.
- **The Telemetry Payload:** Stream payloads monitoring `dispenser_capacity_pct` and `flush_valve_pressure`.
- **The Anomaly Trigger:** Simulate a dispenser capacity dropping below 10% on a heavy-traffic Monday morning.

- **The Expected BMS Action:** The central engine instantly flags a high-priority task on the digital janitorial roster to restock the asset *before* a complaint occurs.

4. THE CORE SOLUTION WORKFLOW (WHAT TO CODE)

Once your data streams hit your backend server, your business logic must process the data through three strict steps:

1. **The Shared Contract Parser:** The server reads incoming data packets from both intern paths seamlessly. Ensure you agree on your JSON object keys together on Day 1.
2. **The Anomaly Engine:** Write a conditional module that scans incoming fields. If any structural error, capacity drop, or environmental failure flag is met, it must instantly mark the agent's record as `[Anomaly Detected]` in the database.
3. **Automated Work Order Generation:** Upon marking an anomaly, your system must programmatically output a unique **Work Order Reference ID**, compute an operational priority level (High/Medium/Low), log an explicit description, and record a timestamp.

5. RESOURCEFULNESS RULE: "USE WHAT YOU HAVE"

As interns executing this project outside the company network, your resourcefulness is a key evaluation metric. You must prove an enterprise concept using absolutely free, accessible alternatives:

- **Instead of expensive mapping APIs (like Google Maps Enterprise):** Use a free open-source alternative like **Leaflet.js**, or simply upload a static JPEG schematic layout of an office floor plan and use basic HTML/CSS absolute positioning coordinates to represent your moving agents.
- **Instead of real hardware sensors:** Use custom loops or simple math functions (`Math.random()`) in your scripts to dynamically generate variable telemetry trends (like dropping battery percentages or changing temperature logs).
- **Instead of cloud networks:** To connect Intern 1's backend server to Intern 2's simulator script over the internet for free, use a tool called **ngrok**. Running `ngrok http 3000` will generate a temporary public URL that lets you pass data between your personal laptops securely.

6. FINAL EVALUATION DEMO CRITERIA

Your internship sprint will culminate in a live screen-share presentation of your local development environments where you demonstrate:

1. **Live Ingestion:** Both background scripts actively pumping distinct operational data into a single, unified local webpage console.
2. **Dashboard Visuals:** Clean visual components showing dropping battery numbers, changing task completion charts, and chronological incident logs.
3. **The Crisis Scenario:** Triggering a simulated everyday error payload live and watching the application instantaneously catch the fault, highlight it in red on the dashboard, and create a fresh database entry with a unique **Work Order Reference ID**